Engenharia de Computação Gráfica e Multimédia

Laboratório de Programação

Jogo Mu Torere

Relatório de Análise e Estruturação da Aplicação

Patrícia Pereira 22304
André Freitas 33782
Grupo: 15
Ano Letivo: 2025/2026

Abril de 2026

Índice

Índice 2
1. Introdução ao Jogo Mu Torere 4
1.1 Origem e Contexto Histórico 4
1.2 Objetivo do Jogo 4
2. Tabuleiro e Componentes 4
3. Regras do Jogo 4
3.1 Regras de Movimento 4
3.2 Condição de Vitória 5
4. Análise de Objetos do Sistema 5
4.1 Objetos Principais 5
5. Ações do Sistema 5
5.1 Ações do Jogador 5
5.2 Ações do Sistema 5
6. Requisitos Funcionais (RF) 5
7. Requisitos Não Funcionais (RNF) 6
8. Fluxo do Jogo 7
9. Visão Geral da Arquitetura 8
10. Pacote Domínio 8
10.1 Classe JOGO 8
10.2 Classe TABULEIRO 9
10.3 Classe CASA 9
10.4 Classe PEÇA 10
10.5 Classe JOGADOR 10
10.6 Enum ESTADO_DE_JOGO 10
10.7 Classe GAMEDEFINITIONS 11
11. Pacote Estado 11
11.1 Classe GAMESTATE 11
11.2 Classe CONFIGURAÇÃO 12
11.3 Classe JOGADA 12
12. Pacote Rede 12
12.1 Classe GAMESERVER 12
12.2 Classe GAMECLIENT 13
12.3 Classe NETWORKMESSAGE 13
12.4 Enum TIPOMENSAGEM 14
12.5 Interface NETWORKLISTENER 14

12.6 Classe CONNECTIONMANAGER 14
13. Pacote Persistência 14
13.1 Classe FILEMANAGER 14
13.2 Enum FORMATOFICHEIRO 15
14. Diagramas UML 15
14.1 Diagrama de Pacotes 15
14.2 Diagrama de Classes Completo 16
14.3 Diagrama de Sequência — Envio de Jogada em Rede 17
14.4 Relações entre Classes 18
15. Conclusão 19

PARTE I

Análise do Jogo Mu Torere

1. Introdução ao Jogo Mu Torere

O Mu Torere é um jogo de tabuleiro tradicional de origem Māori, com forte relevância cultural e histórica.

1.1 Origem e Contexto Histórico

O Mu Torere é originário da costa leste da Ilha Norte da Nova Zelândia. É um dos poucos jogos conhecidos que eram praticados pelos Māori antes da colonização britânica, o que lhe confere elevado valor cultural.

Segundo a tradição, o chefe Tamihana Te Waharoa terá desafiado o governador George Grey para uma partida, colocando simbolicamente o território como aposta — desafio esse que foi recusado.

1.2 Objetivo do Jogo

O objetivo do jogo consiste em bloquear o adversário, impedindo-o de efetuar qualquer movimento válido. Vence o jogador que conseguir imobilizar todas as peças do oponente. Não existem empates.

2. Tabuleiro e Componentes

O tabuleiro é constituído por nove posições: oito periféricas (kewai) e uma central (putahi). Cada jogador controla quatro peças.

Componente	Descrição	Quantidade
Tabuleiro	Estrutura em forma de estrela com 9 posições	**1**
Kewai	Casas periféricas	**8**
Putahi	Casa central	**1**
Peças — Jogador 1	Cor escura	**4**
Peças — Jogador 2	Cor clara	**4**

3. Regras do Jogo

3.1 Regras de Movimento

- Uma peça pode mover-se para um kewai adjacente vazio.
- Uma peça pode mover-se para o putahi apenas se estiver adjacente a pelo menos uma peça adversária.
- Não é permitido saltar peças.

- Não é permitido mover para o putahi se existir um kewai adjacente livre.
- Os jogadores jogam alternadamente, movendo apenas uma peça por turno.

3.2 Condição de Vitória

O jogo termina quando um dos jogadores não possui movimentos válidos. O adversário é declarado vencedor.

4. Análise de Objetos do Sistema

4.1 Objetos Principais

- Jogo: gere o estado global da partida.
- Jogador: representa cada participante.
- Peça: elemento que ocupa uma posição.
- Tabuleiro: estrutura com as posições.
- Posição: unidade do tabuleiro com estado de ocupação.

5. Ações do Sistema

5.1 Ações do Jogador

- Selecionar peça
- Verificar movimentos possíveis
- Executar movimento válido

5.2 Ações do Sistema

- Validar movimentos
- Atualizar tabuleiro
- Verificar condição de vitória
- Alternar turno
- Iniciar nova partida

6. Requisitos Funcionais (RF)

A tabela seguinte apresenta os requisitos funcionais identificados para a aplicação Mu Torere.

ID	Requisito	Descrição
RF01	Iniciar jogo	O sistema deve permitir iniciar uma nova partida com as peças nas posições iniciais.
RF02	Representar tabuleiro	O sistema deve representar corretamente o tabuleiro com 9 posições.
RF03	Mover peça	O sistema deve permitir ao jogador ativo mover uma peça válida por turno.

RF04	Validar movimento	O sistema deve verificar se um movimento é válido antes da sua execução.
RF05	Impedir movimentos inválidos	O sistema deve impedir a execução de movimentos que violem as regras do jogo.
RF06	Alternar turno	O sistema deve alternar automaticamente o turno entre jogadores.
RF07	Verificar condição de vitória	O sistema deve detetar quando um jogador não possui movimentos válidos.
RF08	Declarar vencedor	O sistema deve declarar o vencedor quando o adversário fica bloqueado.
RF09	Interface gráfica (GUI)	O sistema deve apresentar uma interface gráfica interativa com representação do tabuleiro.
RF10	Animação gráfica	O sistema deve apresentar animações dos movimentos das peças.
RF11	Jogo em rede (LAN)	O sistema deve permitir a interação entre dois jogadores através de rede local.
RF12	Enviar jogadas	O sistema deve enviar as jogadas realizadas para o adversário remoto.
RF13	Receber jogadas	O sistema deve receber e aplicar jogadas do adversário remoto.
RF14	Gravar jogo	O sistema deve permitir guardar o estado atual da partida num ficheiro.
RF15	Carregar jogo	O sistema deve permitir carregar uma partida previamente guardada.
RF16	Configurar parâmetros	O sistema deve permitir definir parâmetros da partida (nomes, cores, etc.).

7. Requisitos Não Funcionais (RNF)

Os requisitos não funcionais definem as restrições técnicas e qualitativas da aplicação.

ID	Categoria	Descrição
RNF01	Linguagem	A aplicação deve ser desenvolvida em Java.
RNF02	Interface	A interface deve ser implementada com JavaFX.
RNF03	Usabilidade	A interface deve ser simples, intuitiva e de fácil utilização.
RNF04	Desempenho	O sistema deve responder às ações do utilizador sem latência percetível.
RNF05	Portabilidade	A aplicação deve executar em sistemas com OpenJDK compatível.
RNF06	Persistência	O estado do jogo deve ser armazenado de forma consistente entre execuções.
RNF07	Manutenibilidade	O código deve ser modular, organizado e documentado.

RNF08	Versionamento	O projeto deve utilizar um sistema de controlo de versões (Git).
RNF09	Conformidade	O sistema deve utilizar apenas tecnologias lecionadas na UC.

8. Fluxo do Jogo

1. O sistema inicia a aplicação e apresenta a interface gráfica ao utilizador.
2. O utilizador escolhe o modo de jogo (local ou em rede).
3. No modo em rede:
   - Um jogador inicia a sessão (servidor).
   - O outro jogador liga-se à sessão (cliente).
4. O sistema inicializa o tabuleiro com as peças nas posições iniciais.
5. É definido o jogador que inicia a partida.
6. O jogador ativo seleciona uma peça no tabuleiro.
7. O sistema apresenta os movimentos possíveis para essa peça.
8. O jogador escolhe uma posição de destino.
9. O sistema valida o movimento de acordo com as regras do jogo. Se for inválido, o jogador deve escolher outro movimento.
10. O sistema atualiza o tabuleiro e executa a animação do movimento.
11. No modo em rede, a jogada é enviada para o adversário e o sistema remoto recebe e aplica a jogada.
12. O sistema verifica se o jogador adversário possui movimentos válidos.
13. Se o adversário não tiver movimentos possíveis, o sistema declara o vencedor e a partida termina. Caso contrário, o sistema alterna o turno e o processo repete-se a partir do passo 6.
14. A qualquer momento, o utilizador pode guardar ou carregar uma partida previamente guardada.

PARTE II

Estruturação da Aplicação

9. Visão Geral da Arquitetura

A aplicação Mu Torere em rede está organizada em cinco pacotes com responsabilidades claramente separadas, seguindo o princípio da separação de responsabilidades (SRP). A comunicação entre jogadores é feita via TCP/IP com uma arquitetura cliente-servidor: uma das máquinas atua como servidor e ambas executam o cliente.

Pacote	Classes	Responsabilidade
dominio	Jogo, Tabuleiro, Casa, Peca, Jogador, EstadoJogo, GameDefinitions	Lógica pura do jogo — regras, validações e estado de jogo
estado	GameState, Configuracao, Jogada	Snapshot da partida para sincronização em rede e persistência
rede	GameServer, GameClient, NetworkMessage, TipoMensagem, NetworkListener, ConnectionManager	Comunicação TCP entre os dois jogadores
persistencia	FileManager, FormatoFicheiro	Gravação e carregamento do estado em ficheiro
gui	MainApp, MenuController, GameController, BoardRenderer, ConfigController	Interface gráfica JavaFX (apresentada na fase seguinte)

10. Pacote Domínio

Contém toda a lógica do jogo, independente de rede ou interface. É o núcleo da aplicação — pode ser testado sem GUI ou rede.

10.1 Classe JOGO

Classe central que coordena o fluxo de uma partida. Valida e executa movimentos, verifica a vitória e produz snapshots do estado.

Atributo / Método	Tipo / Retorno	Descrição
- tabuleiro	Tabuleiro	O tabuleiro de jogo com as 9 casas
- jogador1	Jogador	Primeiro participante
- jogador2	Jogador	Segundo participante
- jogadorAtual	Jogador	Referência ao jogador com a vez
- estadoJogo	EstadoJogo	Estado actual da partida (enum)
+ iniciar()	void	Inicializa tabuleiro e distribui peças
+ processarJogada(p,c)	boolean	Valida regras e executa o movimento
+ verificarFimJogo()	boolean	Verifica se adversário tem movimentos
+ mudarTurno()	void	Alterna o jogador activo
+ getEstadoAtual()	GameState	Produz snapshot para serialização/rede
+ restaurarEstado(gs)	void	Restaura partida a partir de GameState

10.2 Classe TABULEIRO

Define a topologia fixa do tabuleiro — as adjacências entre casas são imutáveis e configuradas no construtor com base nas constantes de GameDefinitions.

Atributo / Método	Tipo / Retorno	Descrição
- casas	List<Casa>	Lista com as 9 casas (índice 0 = putahi)
- putahi	Casa	Referência directa à casa central
+ inicializar()	void	Cria 9 casas e define adjacências
+ getCasa(id)	Casa	Retorna Casa com o id indicado
+ getAdjacentes(c)	List<Casa>	Lista de casas adjacentes à Casa c
+ colocarPecas(j1,j2)	void	Distribui as peças na posição inicial
+ toArray()	int[][]	Serializa posições para GameState

10.3 Classe CASA

Atributo / Método	Tipo / Retorno	Descrição
- id	int	Identificador único (0 = putahi, 1-8 = kewai)
- isCentro	boolean	True se for a casa central (putahi)
- pecaOcupante	Peca	Peça na casa — null se vazia
- adjacentes	List<Casa>	Casas directamente ligadas a esta
+ estaVazia()	boolean	Retorna true se pecaOcupante == null
+ ocupar(p)	void	Coloca a peça p nesta casa
+ libertar()	void	Remove a peça (pecaOcupante = null)
+ getAdjacentes()	List<Casa>	Retorna lista de casas adjacentes

10.4 Classe PEÇA

Encapsula a lógica de movimento — sabe calcular as suas próprias casas válidas, aplicando as regras do jogo (adjacência e regra do putahi).

Atributo / Método	Tipo / Retorno	Descrição
- id	int	Identificador único da peça (0-3 por jogador)
- dono	Jogador	Referência ao jogador proprietário
- casaAtual	Casa	Casa onde a peça está actualmente
+ moverPara(c)	void	Move a peça para Casa c
+ getMovimentosValidos()	List<Casa>	Calcula casas para onde pode mover
+ podeMoverPara(c)	boolean	Verifica adjacência e regra do putahi

10.5 Classe JOGADOR

Atributo / Método	Tipo / Retorno	Descrição
- nome	String	Nome do jogador
- pecas	List<Peca>	As 4 peças do jogador
- vezAtual	boolean	Indica se é a vez deste jogador
- cor	String	Cor das peças (hex, ex: "#2E86C1")
+ temMovimentosPossiveis()	boolean	Verifica se alguma peça pode mover
+ getPecas()	List<Peca>	Retorna a lista de peças
+ setVez(b)	void	Define se é ou não a vez deste jogador
+ toString()	String	Nome + cor para debug/logging

10.6 Enum ESTADO_DE_JOGO

Valor	—	Descrição
A_DECORRER	–	Partida em curso — há movimentos disponíveis
VITORIA_J1	–	Jogador 1 venceu — J2 sem movimentos
VITORIA_J2	–	Jogador 2 venceu — J1 sem movimentos
EMPATE	–	Nenhum jogador tem movimentos (variante rara)

10.7 Classe GAMEDEFINITIONS

Classe de constantes e utilitários do domínio — centraliza os valores fixos do jogo para evitar magic numbers dispersos pelo código.

Atributo / Método	Tipo / Retorno	Descrição
+ NUM_PECAS	int = 4	Número de peças por jogador
+ NUM_CASAS	int = 9	Total de casas no tabuleiro
+ PUTAHI_ID	int = 0	Índice da casa central
+ getAdjacencias()	int[][]	Mapa fixo de adjacências entre casas
+ getPosicaoInicial(jogadorId)	int[]	Kewai iniciais de cada jogador

11. Pacote Estado

Contém as classes que representam o estado serializável da aplicação — usadas tanto para envio pela rede como para persistência em ficheiro.

11.1 Classe GAMESTATE

Snapshot completo e imutável da partida num dado momento. Implementa Serializable para poder ser enviado via ObjectOutputStream e gravado em ficheiro. É o único objeto que circula pela rede.

Atributo / Método	Tipo / Retorno	Descrição
- tabuleiro	int[][]	Posição de cada peça por id de casa
- jogadorAtualId	int	Id do jogador com a vez (0 ou 1)
- turno	int	Número do turno actual
- estadoJogo	EstadoJogo	Estado actual da partida
- historico	List<Jogada>	Histórico de todas as jogadas
+ snapshot(jogo)	GameState	Cria snapshot a partir do Jogo actual
+ restaurar()	Jogo	Reconstrói o Jogo a partir deste estado
+ toJson()	String	Serializa para JSON (alternativa a .ser)
+ fromJson(s)	GameState	Desserializa de String JSON

11.2 Classe CONFIGURAÇÃO

Agrupa todos os parâmetros configuráveis pelo utilizador. É carregada no arranque e guardada ao fechar a aplicação.

Atributo / Método	Tipo / Retorno	Descrição
- nomeJ1 / nomeJ2	String	Nomes dos dois jogadores
- corJ1 / corJ2	String	Cores das peças em hex (ex: "#2E86C1")
- enderecoServidor	String	IP ou hostname do servidor
- porta	int	Porta TCP (padrão: 12345)
- tempoLimiteSeg	int	Tempo máximo por jogada (0 = sem limite)
+ carregar(ficheiro)	Configuracao	Carrega de ficheiro .properties
+ guardar(ficheiro)	void	Persiste configuração em ficheiro
+ getDefaults()	Configuracao	Retorna configuração com valores padrão

11.3 Classe JOGADA

Registo imutável de um único movimento. Compõe o histórico dentro do GameState.

Atributo / Método	Tipo / Retorno	Descrição
- pecaId	int	Id da peça movida
- casaOrigemId	int	Id da casa de origem
- casaDestinoId	int	Id da casa de destino
- turno	int	Número do turno em que ocorreu
- timestamp	long	Timestamp em milissegundos
+ toString()	String	Representação legível para logging

12. Pacote Rede

Implementa a comunicação TCP entre as duas instâncias da aplicação. A arquitetura é cliente-servidor: uma máquina inicia o GameServer e ambas executam um GameClient. As mensagens trafegam como objetos NetworkMessage via ObjectOutputStream.

12.1 Classe GAMESERVER

Corre numa das máquinas (Jogador 1 ou servidor dedicado). Aceita ligações, recebe jogadas de um cliente e difunde o novo estado a todos.

Atributo / Método	Tipo / Retorno	Descrição
- serverSocket	ServerSocket	Socket de escuta de ligações
- clientes	List<Socket>	Sockets dos clientes ligados
- jogo	Jogo	Instância do jogo gerida pelo servidor
- porta	int	Porta TCP de escuta
+ iniciar(porta)	void	Abre o ServerSocket e fica à escuta
+ aguardarLigacoes()	void	Thread que aceita novos clientes
+ difundir(msg)	void	Envia NetworkMessage a todos os clientes
+ processarMensagem(msg)	void	Interpreta e aplica a mensagem recebida
+ parar()	void	Fecha todos os sockets graciosamente

12.2 Classe GAMECLIENT

Instanciado em cada máquina. Liga-se ao servidor, envia jogadas e recebe atualizações de estado que notifica à GUI via NetworkListener.

Atributo / Método	Tipo / Retorno	Descrição
- socket	Socket	Ligação TCP ao servidor
- in	ObjectInputStream	Stream de leitura de objectos
- out	ObjectOutputStream	Stream de escrita de objectos
- listener	NetworkListener	Callback para notificar a GUI
+ ligar(ip, porta)	void	Estabelece ligação TCP ao servidor
+ enviarJogada(j)	void	Envia Jogada encapsulada em NetworkMessage
+ enviarEstado(gs)	void	Envia GameState ao servidor
+ desligar()	void	Fecha socket e streams graciosamente
+ ouvirMensagens()	void	Thread de leitura contínua de mensagens

12.3 Classe NETWORKMESSAGE

Envelope que encapsula qualquer mensagem trocada pela rede. Implementa Serializable. O campo payload transporta o objeto concreto (Jogada, GameState, etc.).

Atributo / Método	Tipo / Retorno	Descrição
- tipo	TipoMensagem	Identifica o tipo de mensagem (enum)
- payload	Object	Objecto concreto transportado (Serializable)
- remetenteId	int	Id do jogador remetente (0 ou 1)
- timestamp	long	Timestamp de criação da mensagem
+ getTipo()	TipoMensagem	Retorna o tipo da mensagem
+ getPayload()	Object	Retorna o payload — cast pelo receptor

12.4 Enum TIPOMENSAGEM

Valor	—	Quando é usado
JOGADA	–	Envio de uma Jogada do cliente para o servidor
ESTADO_JOGO	–	Difusão do GameState actualizado a ambos os clientes
INICIO_JOGO	–	Sinaliza que ambos os clientes estão prontos
FIM_JOGO	–	Partida terminada — inclui id do vencedor no payload
DESLIGADO	–	Um cliente desligou-se inesperadamente
PING	–	Keepalive para verificar se a ligação está activa

12.5 Interface NETWORKLISTENER

Contrato que o GameController (GUI) implementa para receber notificações assíncronas do GameClient.

Método	Retorno	Quando é invocado
onMensagemRecebida(m)	void	Chegou uma NetworkMessage do servidor
onConexaoEstabelecida()	void	Ligação TCP confirmada com sucesso
onDesconexao()	void	Ligação perdida ou adversário desligou
onErro(e)	void	Erro de I/O na comunicação de rede

12.6 Classe CONNECTIONMANAGER

Gere o ciclo de vida da ligação TCP — tentativas de ligação inicial, reconexões automáticas em caso de falha e timeouts.

Atributo / Método	Tipo / Retorno	Descrição
- cliente	GameClient	Instância do cliente a gerir
- reconectando	boolean	Flag de reconexão em curso
- maxTentativas	int	Máximo de tentativas de reconexão
+ conectar(ip, porta)	void	Tenta estabelecer ligação
+ reconectar()	void	Tenta reconnectar com backoff exponencial
+ isConectado()	boolean	Verifica se a ligação está activa

13. Pacote Persistência

Responsável por gravar e carregar o estado completo da aplicação — tabuleiro, jogador atual e configurações — num ficheiro local.

13.1 Classe FILEMANAGER

Abstrai toda a lógica de I/O. Suporta dois formatos: serialização Java binária (.ser) e JSON (.json), configurável via FormatoFicheiro.

Atributo / Método	Tipo / Retorno	Descrição
- caminhoFicheiro	String	Caminho base para ficheiros de jogo
- formato	FormatoFicheiro	Formato de serialização (enum)
+ guardar(gs)	void	Serializa GameState para ficheiro
+ carregar(ficheiro)	GameState	Lê e desserializa GameState
+ guardarConfig(c)	void	Persiste objecto Configuracao
+ carregarConfig()	Configuracao	Carrega Configuracao do disco
+ ficheiroExiste(f)	boolean	Verifica se um ficheiro de jogo existe
+ listarGuardados()	List<String>	Lista ficheiros de jogo disponíveis

13.2 Enum FORMATOFICHEIRO

Valor	—	Descrição
SERIALIZADO	–	Serialização binária Java (.ser) via ObjectOutputStream
JSON	–	Formato texto legível (.json) via toJson/fromJson de GameState

14. Diagramas UML

14.1 Diagrama de Pacotes

Visão de alto nível dos cinco pacotes e as suas dependências. As setas a tracejado representam dependências de uso (um pacote usa classes de outro).

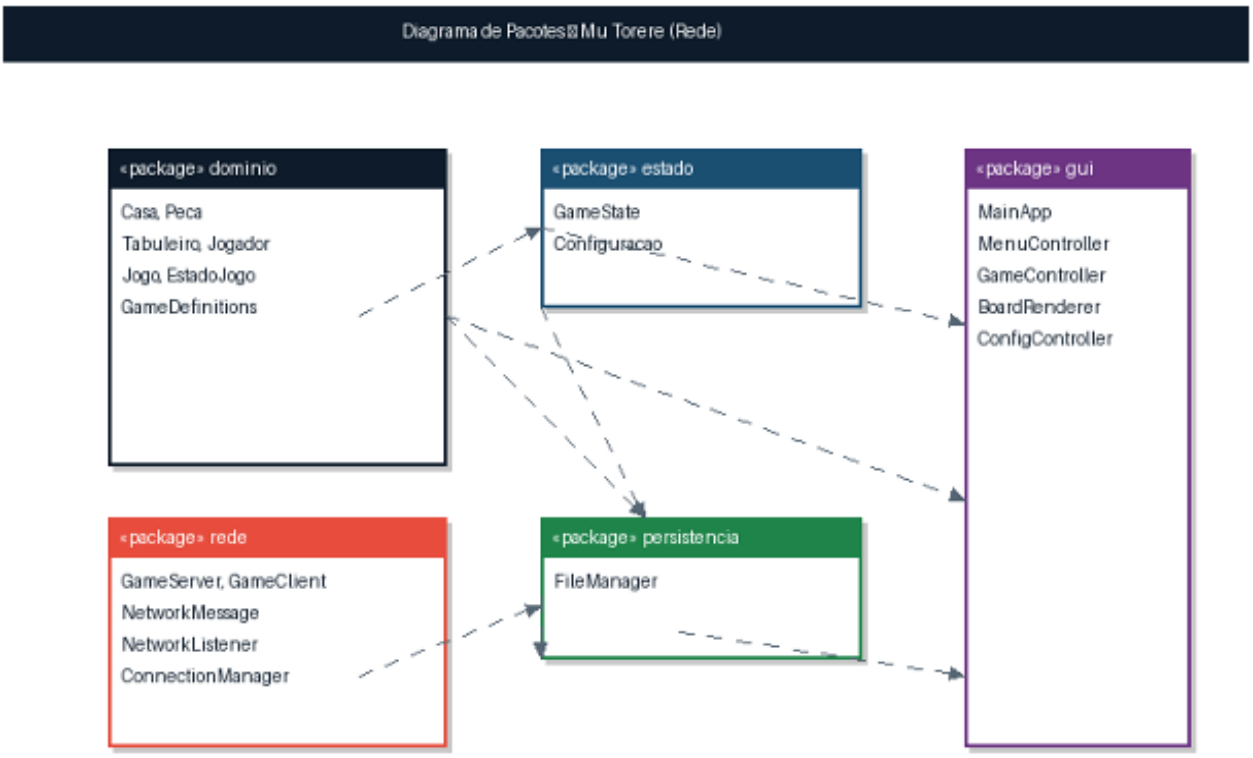

Figura 1 — Diagrama de Pacotes da Aplicação Mu Torere

14.2 Diagrama de Classes Completo

O diagrama seguinte representa todas as classes, organizadas por pacote (strip colorida no topo de cada caixa), com atributos e métodos. As setas sólidas representam associações/composições; as tracejadas representam dependências.

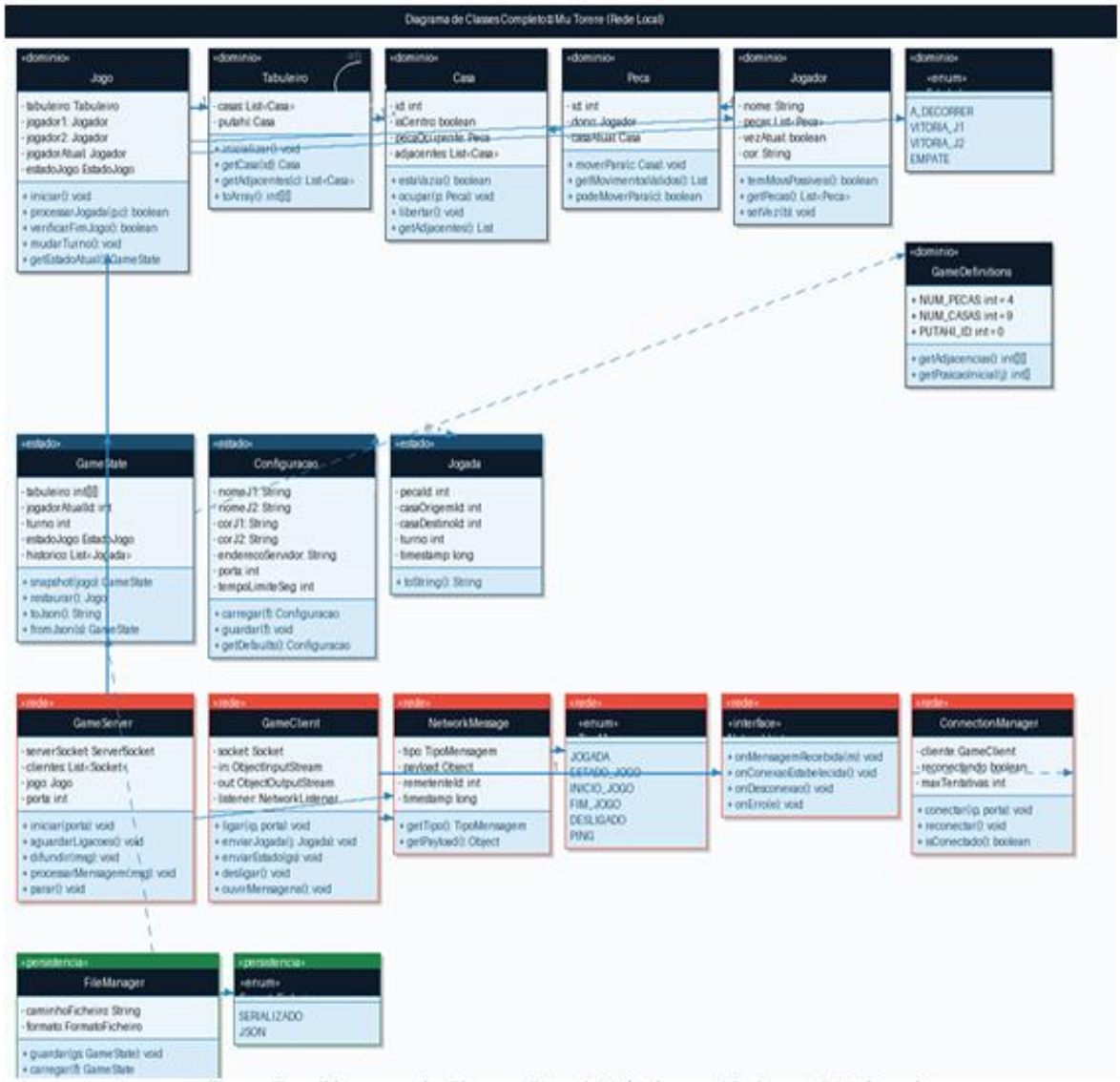

Figura 2 — Diagrama de Classes Completo (todas as 18 classes/interfaces)

14.3 Diagrama de Sequência — Envio de Jogada em Rede

Ilustra o fluxo de mensagens quando o Jogador 1 realiza uma jogada: o GameClient local envia a Jogada ao GameServer, que valida, atualiza o GameState e difunde o novo estado a ambos os clientes.

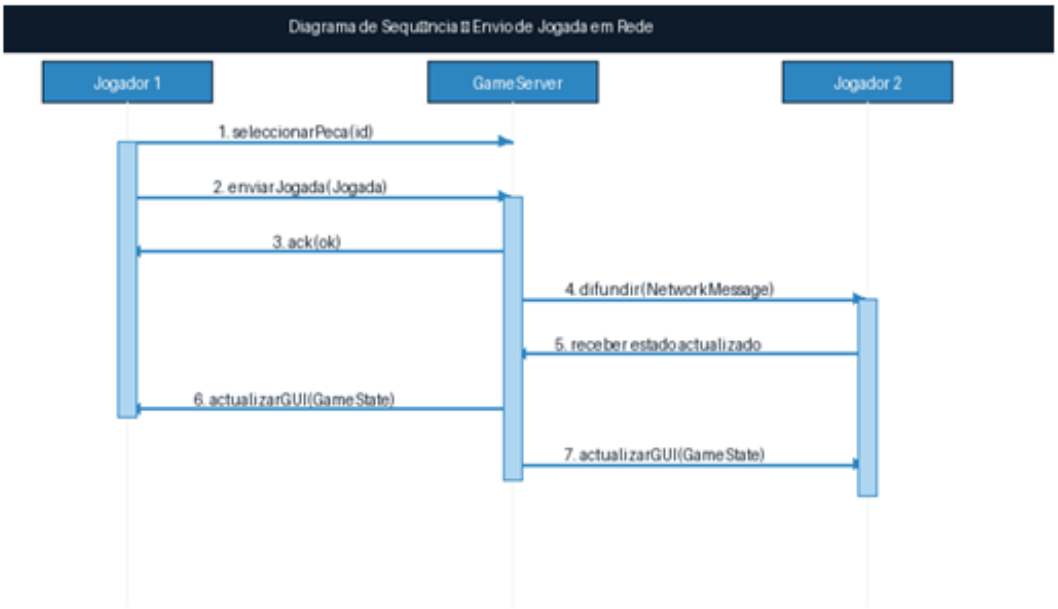

Figura 3 — Diagrama de Sequência de uma Jogada em Rede

14.4 Relações entre Classes

Relação	Tipo	Descrição
Jogo → Tabuleiro	Composição (1:1)	O Jogo possui e cria o Tabuleiro
Jogo → Jogador	Composição (1:2)	O Jogo gere exactamente 2 Jogadores
Jogo → EstadoJogo	Associação (1:1)	O Jogo mantém o estado actual
Tabuleiro → Casa	Composição (1:9)	O Tabuleiro compõe-se de 9 Casas
Casa → Casa	Associação reflexiva	Cada Casa conhece as suas adjacentes
Jogador → Peca	Composição (1:4)	Cada Jogador possui exactamente 4 Peças
Peca → Casa	Associação (1:1)	Cada Peça está posicionada numa Casa
GameState → Jogada	Agregação (1:*)	GameState contém histórico de Jogadas
GameState → EstadoJogo	Associação	GameState regista o estado da partida
GameServer → Jogo	Associação (1:1)	O servidor controla a instância do Jogo
GameClient → NetworkListener	Associação (1:1)	Notifica a GUI via interface
NetworkMessage → TipoMensagem	Associação (1:1)	Classifica o tipo de mensagem
ConnectionManager → GameClient	Composição (1:1)	Gere o ciclo de vida do cliente
FileManager → GameState	Dependência	Serializa/desserializa o GameState
FileManager → Configuracao	Dependência	Persiste/carrega a Configuracao
GameController → NetworkListener	Realização	GUI implementa a interface

15. Conclusão

O presente relatório documenta de forma integrada as duas fases do desenvolvimento da aplicação Mu Torere: a análise do jogo e a estruturação técnica da solução implementada.

Na fase de análise, foi feita uma caracterização aprofundada do jogo Mu Torere — um jogo de tabuleiro de origem Māori com valor cultural e histórico assinalável. Foram identificados os seus componentes, regras de movimento e condição de vitória, bem como os objetos do sistema e as interações esperadas. Com base nessa análise, foram definidos 16 requisitos funcionais e 9 requisitos não funcionais, que guiaram as escolhas de desenho e implementação. O fluxo do jogo foi igualmente formalizado, cobrindo tanto o modo local como o modo em rede.

Na fase de estruturação, a aplicação foi organizada em cinco pacotes com responsabilidades bem delimitadas: domínio, estado, rede, persistência e interface. Esta separação garante uma arquitetura modular, testável e manutenível. O pacote de domínio encapsula toda a lógica do jogo de forma independente da rede ou da GUI, enquanto o pacote de rede implementa uma arquitetura TCP/IP cliente-servidor que permite a comunicação assíncrona entre dois jogadores em rede local. O pacote de estado assegura a serialização e partilha do estado de jogo, e o pacote de persistência suporta a gravação e carregamento de partidas em múltiplos formatos.

Os diagramas UML produzidos — de pacotes, de classes, de sequência e de relações — confirmam a coerência da arquitetura proposta e a clareza das dependências entre os seus componentes.

Em suma, a solução desenvolvida cumpre os requisitos funcionais e não funcionais definidos, respeitando as tecnologias lecionadas na unidade curricular. A arquitetura modular adotada facilita a manutenção futura, a adição de novas funcionalidades e a reutilização de componentes, constituindo uma base sólida para o desenvolvimento contínuo do projeto.